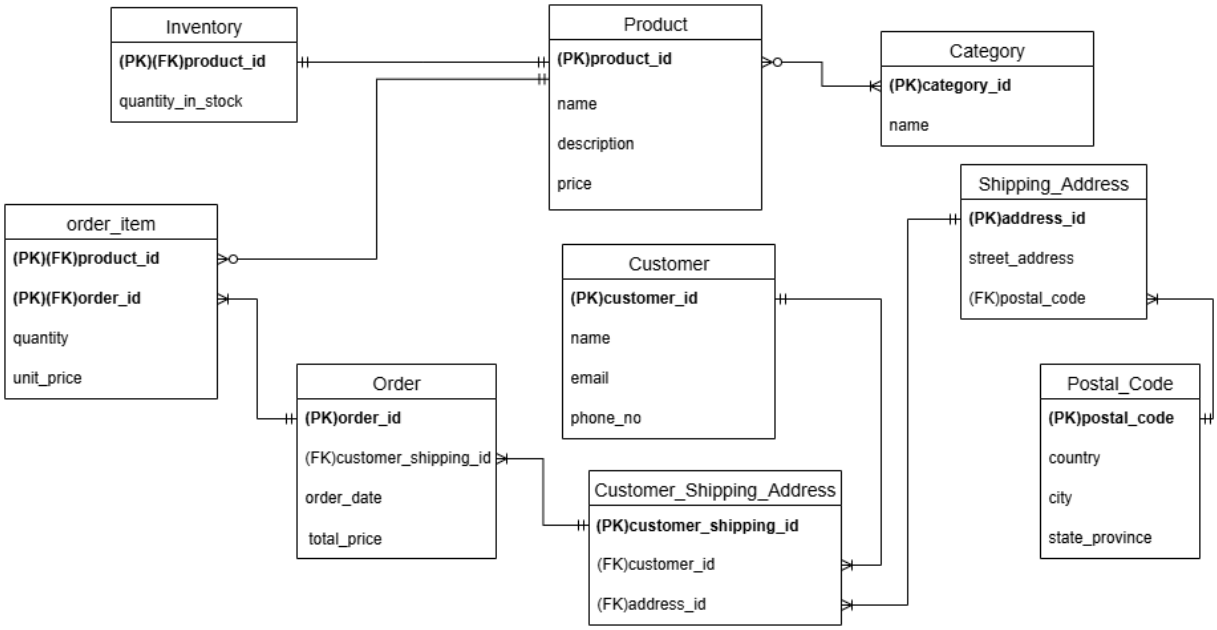
Assignment DB - Relational Database Design and Optimization

1. Assumptions

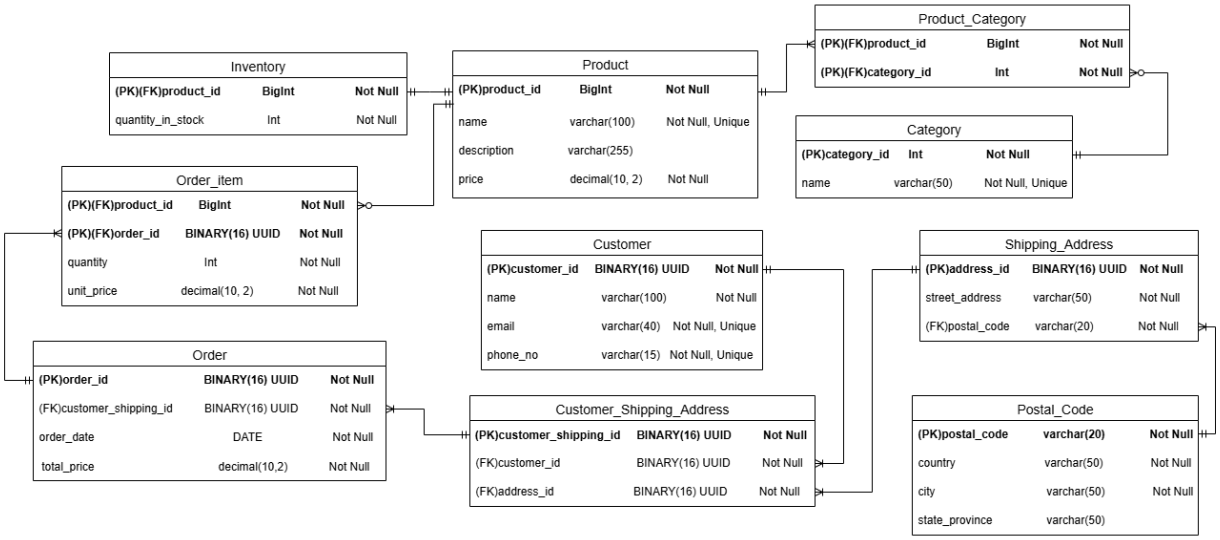
- Product to Category Relationship(M: N) - A Product can be categorized into one or many categories. A category can have zero or many products. (eg, TV categories – Electronics, Home Appliances)
- Order Item Entity – Unit price means the price of that item when it is ordered. Using the price attribute of the product entity for calculations can give incorrect outputs. Because the price attribute of the Product entity can be changed in future.
- Customer to Shipping Address Relationship(M: N) – Customer can have different shipping addresses. And there can be situations where each member of the same house(same address) uses the same address.
- Postal Code Entity – Country, city, and state_province attributes are dependent on the postal code. That is why it has a separate entity.

2. Logical ERD



3. Relational Schema

- Join tables are included in between M: N relationships(Product to Category, Customer to Shipping Address).
- BINARY(16) UUID is used for primary keys of entities that can grow hugely in the future and need more security.
- Use the int data type for the Category entity primary key since its categories do not grow as much as compared to others.
- Use BigInt as the Product primary key. Because having sequential, predictable IDs are sufficient for this entity and does not require high security compared to others that used UUID.



4. Database Optimization

4.1. Indexing

Creating proper Indexes helps to speed up the search and filter operations in the database. When creating the tables, primary keys and unique keys have indexes automatically.

1. Examples

- product_id in Product and order_id in Orders are primary keys, and they automatically have indexes. This allows MySQL to instantly locate products or orders without scanning the entire table.
- Also, unique constraints such as UC_Customer_Email create an index on email in the customer table, making customer lookups by email fast.
- In order-item, order_id references Orders(order_id) and product_id references Product(product_id). MySQL automatically creates indexes on these columns. This makes joins very fast, because MySQL doesn't need to scan all rows. It uses an index to directly find the matching rows.

Without an index, DB must scan the entire table to find the correct row and retrieve the data. Therefore, it becomes slow when the database grows. But with an index, DB can directly find the relevant row to retrieve the data.

4.2. Normalization

- Normalization is mainly used to avoid complexities, reduce redundancy, and maintain the integrity of the database.
- Therefore, databases become easier to maintain, update, and provide fast data retrieval since the database storage is smaller.
- The entire database is going through 1NF, 2NF, and 3NF steps to normalize the database.
- 1 NF – all the cells in each table have only one value.
- 2 NF – All non-key attributes should be fully dependent on a primary key. After the 1 NF, the database is already in the 2nd NF since all the attributes fully depend on the specific table's Primary Key.
- 3 NF – Non-Prime attribute is dependent on another non-prime attribute. In the Shipping Address table, country, city, state/province attributes are dependent on the postal code. Therefore, a table wasn't in the 3rd NF. To pass the 3rd NF, another table is created, named Postal Code, including country, city, state/province, and postal code, which becomes its primary Key.

4.3. Use of proper data types

- Use the proper data types to reduce the database storage size, security, and data accuracy.
- UUIDs (Universally unique identifiers) are stored using binary(16) to reduce the storage used to store primary keys.
- And for prices such as total prices and sub prices, decimal(10, 2) is used to ensure exact calculations without floating point errors.

4.4. Use of proper constraints

- Use Proper Constraints for the attributes, such as UNIQUE, CHECK, and NOTNULL, to guard and protect at the database level to enforce rules that can be missed by the application code.
- Example: Customer Emails, phone numbers, Category, and product name are configured as Unique.
- Add a Not Null constraint for each attribute that should not have empty cells.
- Add a Check(>0) constraint to attributes such as quantity in the stock and product quantity in the orders, to make sure quantities are not lower than 0.

5. Database Security

5.1.1. Database roles and users' creation

- Initially, the root user, which is created in the installation process, is used to execute all the queries and configurations in the database.
- Root users have the highest level of permission within the MySQL environment.
- Having only the root user is enough if the database is not that huge, distributed, or if it is created for a short period of time. But it is not secure enough if the database becomes larger over time.
- There can be security issues allowing all the users to log in as root users, because multiple users having all the administrative permissions can be an issue.
- Therefore, several roles can be created with different permissions to execute SQL Queries. And then database users can be created by assigning those roles, including a username and a password to log in.
- Therefore, each role can only have granted permission.
- Roles have collection permissions that can be assigned to each user.
- For shop4All DB, 4 roles were created in addition to the root.

```
-- Roles Creation --
create role 'primary_role'@'localhost';
create role 'reporting_role'@'localhost';
create role 'inventory_manager_role'@'localhost';
create role 'order_manager_role'@'localhost';
```

Figure 5-1: Role Creation

- Also, all these roles can be used only when the user is connecting though the localhost(connecting through the same machine MYSQL is running).
- Then different permissions are granted to above rules according to this database.

```
-- primary roles only can have  select, insert, update all the databases
grant select, insert, update on shop4Alldb.*
to 'primary_role'@'localhost';

-- reporting roles only can have select and retrive all the data from the databases
grant select on shop4Alldb.*
to 'reporting_role'@'localhost';

-- inventory roles only can execute crud operations to the product and inventory tables
grant select, update, insert, delete on shop4Alldb.product
to 'inventory_manager_role'@'localhost';
grant select, update, insert, delete on shop4Alldb.Inventory
to 'inventory_manager_role'@'localhost';

-- order_manager_role roles only can execute crud operations to the order and order item tables
grant select, update, insert, delete on shop4Alldb.orders
to 'order_manager_role'@'localhost';
grant select, update, insert, delete on shop4Alldb.order_item
to 'order_manager_role'@'localhost';
```

Figure 5-2: Granting Role Permissions

- Then, database users are created with different usernames and passwords to log in. MySQL validates credentials on login.
- Then, based on the user, different roles are assigned. As an example, if a reporting user is logged, he can only execute select commands for any database. And if he tries to execute other commands other than select, it will give an error.

```
-- creating users
create user 'primary_user_1'@'localhost' identified by 'primary123@#';
create user 'reporting_user_1'@'localhost' identified by 'reporter123@#';
create user 'inventory_manager_user_1'@'localhost' identified by 'inventory123@#';
create user 'order_manager_1'@'localhost' identified by 'order123@#';

-- Assigning roles to users
grant 'primary_role'@'localhost' to 'primary_user_1'@'localhost';
grant 'reporting_role'@'localhost' to 'reporting_user_1'@'localhost';
grant 'inventory_manager_role'@'localhost' to 'inventory_manager_user_1'@'localhost';
grant 'order_manager_role'@'localhost' to 'order_manager_1'@'localhost';
```

Figure 5-3: DB users' creation

5.1.2. Using Cascade on Parent and Child Tables

Using Cascade Restrict

There can be situations in the database that should not allow the deletion of a row. As an example, when a product is deleted, the relevant order items(products) from the order items table should not be deleted. Because if it's deleted, there can be an issue in the order history since several order-items(records) are deleted from the table. So, the data will be inaccurate.

To restrict this issue, Cascade restrict is used when creating a foreign key for the order-item table.

```
create table Order_Item(
order_id binary(16) NOT NULL,
product_id bigint unsigned NOT NULL,
quantity int NOT NULL check (quantity>0),
unit_price decimal(10, 2) NOT NULL,
constraint PK_Order_Product PRIMARY KEY (order_id, product_id),
constraint FK_Order_Product_Order FOREIGN KEY (order_id) references Orders(order_id) on update cascade on delete cascade,
constraint FK_Order_Product_Product FOREIGN KEY (product_id) references Product(product_id) on update cascade on delete restrict
);
```

Figure 5-4: Using Cascade

Here, when a user tries to delete products, cascade restrict is used on delete operation when creating the foreign key for the product table.

5.1.3. Data Protection Using UUIDs.

Use the UUID with binary(16) to store ids of tables which are needed more security compared to others such as Orders, Customers, and Customer_Shipping_Address. Since

UUIDs are unique and less human readable, it is better to use to protect data from unauthorized access.

5.1.4. Using Transactions

Used transactions for operations that should be executed one after the other to allow data to be accurate in the database.

Examples: Adding new Orders – When order is placed, order-item table, and inventory table should be updated based on the quantity of the products in the order.

6. Use of transactions to guarantee ACID Principle of database operations.

Transactions are used for group-related operations together, such as adding a product, placing an order, or adding a new shipping address. Transactions ensure that databases always follow the ACID principle.

- Atomicity - All the operations of the transaction are completed or all fail(Rollback). As an example, when placing an order, data is inserted into the order table, then into the order items table, and it should update the inventory table. With the transactions, all these operations are done, or all will fail(Rollback) without having partially succeeded, such as having an order without any order-items or inventory quantity reduction.
- Consistency - A transaction moved the database from one consistent state to another by respecting all rules, constraints, and relationships. If one of the rules is broken, the entire transaction will roll back. As an example, if an invalid customer ID is added in the shipping address transaction, the foreign key constraint is violated, and the whole transaction will fail. Therefore, DB will not be in an invalid state.
- Isolation - Transactions running concurrently do not affect each other's output. As an example, if two customers order the last product, both actions won't succeed. Because MySQL uses row-level locks. It means that while one transaction is updating the inventory, the other should wait until the first one is completed. Therefore, only one order will get the last product, preventing double-selling.
- Durability - Once a transaction is completed, its changes are permanently stored in the database. If the system crashes immediately after placing an order, it will still exist when the database restarts. This ensures that important data like orders, inventory, or order items are never lost after a commit.